

INDUSTRY PROBLEM-SOLVING TASK

Java AWT, Applets, Event Handling and Layout Managers

Name of the Student	J. Renuka Devi	Register No.	192573007
Programme / Section	BE. CSE-DS	Date	22.08.2026
Course Code	CSA0925	Course	Programming in Java
Faculty	Dr. C. Sivasankar	Duration	Lab Session

INDUSTRY TASK 2: Online Food Ordering and Billing System

Problem Statement: A restaurant wants to develop a simple Food Ordering and Billing Application for its counter staff. The application should allow customers to select food items, specify quantities, and generate a final bill.

Functional Requirements

- Develop the application using Java AWT.
- Create fields for Customer Name and Table Number.
- Provide Food Category, Food Item, and Quantity selection using appropriate AWT components.
- Provide Add Item, Generate Bill, Clear, Next, and Previous buttons.
- Display selected food items in a List or TextArea.
- Calculate item-wise cost, Subtotal, GST, and Final Bill Amount.
- Implement button click events using ActionListener.
- Use CardLayout to create separate screens for Customer/Order Details, Food Selection, and Bill/Order Summary.
- Validate quantity and ensure that an item is selected before adding it to the order.

Industry Context: The application represents a simplified restaurant POS (Point of Sale) system used for taking customer orders, processing prices, and generating a customer bill.

Expected Deliverables

- Java source code.
- Working multi-card AWT interface.

- Sample order and generated bill.
- Screenshot of each card/screen.
- Brief explanation of CardLayout, AWT components, and event handling.

Student Instructions

1. The solution must be implemented in Java using the concepts covered in the laboratory syllabus.
2. Students should demonstrate compilation, execution, and output during the practical session.
3. Use meaningful variable names and appropriate comments.
4. Students must be able to explain the selected AWT components, layout manager, event handling mechanism, and processing logic.
5. Evaluation Rubric

Criterion	Weightage	Marks / Remarks
Problem Understanding	15%	
GUI Components and Layout	20%	
Event Handling	20%	
Functional Correctness	25%	
Input Validation and User Experience	10%	
Program Explanation / Viva	10%	

6.

Java Code:

```
import java.awt.*;
import java.awt.event.*;
import java.util.ArrayList;
public class FoodOrderingSystem extends Frame implements ActionListener {
    CardLayout cardLayout;
    Panel cardPanel;
    TextField customerNameField;
    TextField tableNumberField;
    Button nextButton;
    Choice categoryChoice;
    Choice foodChoice;
    TextField quantityField;
    Button addButton;
    Button generateBillButton;
    Button previousButton;
    TextArea billArea;
    Button clearButton;
    Button billPreviousButton;
```

```
ArrayList<String> items = new ArrayList<>();
ArrayList<Integer> quantities = new ArrayList<>();
ArrayList<Double> prices = new ArrayList<>();
```

```
String[] foodNames = {
    "Burger",
    "Pizza",
    "Pasta",
    "Fried Rice",
    "Noodles",
    "Sandwich",
    "Ice Cream",
    "Coffee"
};
```

```
double[] foodPrices = {
    120,
    250,
    180,
    160,
    150,
    100,
    80,
    60
};
```

```
public FoodOrderingSystem() {

    setTitle("Online Food Ordering & Billing System");
    setSize(700, 600);

    cardLayout = new CardLayout();
    cardPanel = new Panel(cardLayout);

    createCustomerCard();
    createFoodCard();
    createBillCard();

    add(cardPanel);

    addWindowListener(new WindowAdapter() {
        public void windowClosing(WindowEvent e) {
            dispose();
        }
    });
}
```

```

    }
});

setVisible(true);
}

private void createCustomerCard() {
    Panel panel = new Panel(new GridBagLayout());

    GridBagConstraints gbc = new GridBagConstraints();

    gbc.insets = new Insets(10, 10, 10, 10);
    gbc.fill = GridBagConstraints.HORIZONTAL;

    Label title = new Label(
        "CUSTOMER / ORDER DETAILS",
        Label.CENTER
    );

    title.setFont(new Font("Arial", Font.BOLD, 22));

    gbc.gridx = 0;
    gbc.gridy = 0;
    gbc.gridwidth = 2;

    panel.add(title, gbc);

    gbc.gridwidth = 1;

    Label nameLabel = new Label("Customer Name:");
    customerNameField = new TextField(20);

    gbc.gridx = 0;
    gbc.gridy = 1;
    panel.add(nameLabel, gbc);

    gbc.gridx = 1;
    panel.add(customerNameField, gbc);

    Label tableLabel = new Label("Table Number:");
    tableNumberField = new TextField(20);

    gbc.gridx = 0;
    gbc.gridy = 2;

```

```

panel.add(tableLabel, gbc);

gbc.gridx = 1;
panel.add(tableNumberField, gbc);

nextButton = new Button("Next");

nextButton.addActionListener(this);

gbc.gridx = 1;
gbc.gridy = 3;

panel.add(nextButton, gbc);

cardPanel.add(panel, "CUSTOMER");
}

private void createFoodCard() {

    Panel panel = new Panel(new GridBagLayout());

    GridBagConstraints gbc = new GridBagConstraints();

    gbc.insets = new Insets(8, 8, 8, 8);
    gbc.fill = GridBagConstraints.HORIZONTAL;

    Label title = new Label(
        "FOOD SELECTION",
        Label.CENTER
    );

    title.setFont(new Font("Arial", Font.BOLD, 22));

    gbc.gridx = 0;
    gbc.gridy = 0;
    gbc.gridwidth = 2;

    panel.add(title, gbc);

    gbc.gridwidth = 1;

    // Category
    Label categoryLabel = new Label("Food Category:");

```

```

categoryChoice = new Choice();

categoryChoice.add("Main Course");
categoryChoice.add("Fast Food");
categoryChoice.add("Beverages");
categoryChoice.add("Dessert");

gbc.gridx = 0;
gbc.gridy = 1;
panel.add(categoryLabel, gbc);

gbc.gridx = 1;
panel.add(categoryChoice, gbc);

// Food item
Label foodLabel = new Label("Food Item:");

foodChoice = new Choice();

for (String food : foodNames) {
    foodChoice.add(food);
}

gbc.gridx = 0;
gbc.gridy = 2;
panel.add(foodLabel, gbc);

gbc.gridx = 1;
panel.add(foodChoice, gbc);

// Quantity
Label quantityLabel = new Label("Quantity:");

quantityField = new TextField(10);

gbc.gridx = 0;
gbc.gridy = 3;
panel.add(quantityLabel, gbc);

gbc.gridx = 1;
panel.add(quantityField, gbc);

// Buttons
Panel buttonPanel = new Panel();

```

```

addButton = new Button("Add Item");
generateBillButton = new Button("Generate Bill");
previousButton = new Button("Previous");

buttonPanel.add(addButton);
buttonPanel.add(generateBillButton);
buttonPanel.add(previousButton);

addButton.addActionListener(this);
generateBillButton.addActionListener(this);
previousButton.addActionListener(this);

gbc.gridx = 0;
gbc.gridy = 4;
gbc.gridwidth = 2;

panel.add(buttonPanel, gbc);

Label orderLabel = new Label("Current Order:");

gbc.gridx = 0;
gbc.gridy = 5;
gbc.gridwidth = 2;

panel.add(orderLabel, gbc);

cardPanel.add(panel, "FOOD");
}
private void createBillCard() {

    Panel panel = new Panel(new BorderLayout(10, 10));

    Label title = new Label(
        "BILL / ORDER SUMMARY",
        Label.CENTER
    );

    title.setFont(new Font("Arial", Font.BOLD, 22));

    panel.add(title, BorderLayout.NORTH);

    billArea = new TextArea();
    billArea.setEditable(false);

```

```

panel.add(billArea, BorderLayout.CENTER);

Panel buttonPanel = new Panel();

billPreviousButton = new Button("Previous");
clearButton = new Button("Clear");

billPreviousButton.addActionListener(this);
clearButton.addActionListener(this);

buttonPanel.add(billPreviousButton);
buttonPanel.add(clearButton);

panel.add(buttonPanel, BorderLayout.SOUTH);

cardPanel.add(panel, "BILL");
}
@Override
public void actionPerformed(ActionEvent e) {

    // NEXT
    if (e.getSource() == nextButton) {

        String name = customerNameField.getText().trim();
        String table = tableNumberField.getText().trim();

        if (name.isEmpty() || table.isEmpty()) {

            showMessage(
                "Please enter Customer Name and Table Number."
            );

            return;
        }

        try {

            int tableNumber = Integer.parseInt(table);

            if (tableNumber <= 0) {

                showMessage(
                    "Table Number must be greater than 0."
                );
            }
        }
    }
}

```



```

        );

        return;
    }

    cardLayout.show(cardPanel, "FOOD");

} catch (NumberFormatException ex) {

    showMessage(
        "Table Number must be numeric."
    );
}
}

else if (e.getSource() == addButton) {

    String quantityText =
        quantityField.getText().trim();

    if (quantityText.isEmpty()) {

        showMessage("Enter quantity.");
        return;
    }

    try {

        int quantity =
            Integer.parseInt(quantityText);

        if (quantity <= 0) {

            showMessage(
                "Quantity must be greater than 0."
            );

            return;
        }

        int index =
            foodChoice.getSelectedIndex();

        String food =

```

```

        foodChoice.getSelectedItem();

        double price =
            foodPrices[index];

        items.add(food);
        quantities.add(quantity);
        prices.add(price);

        showMessage(
            food + " added to order."
        );

        quantityField.setText("");

    } catch (NumberFormatException ex) {

        showMessage(
            "Quantity must be a valid number."
        );
    }
}

// GENERATE BILL
else if (e.getSource() == generateBillButton) {

    if (items.size() == 0) {

        showMessage(
            "Please add at least one item."
        );

        return;
    }

    generateBill();

    cardLayout.show(cardPanel, "BILL");
}

// FOOD PREVIOUS
else if (e.getSource() == previousButton) {

    cardLayout.show(cardPanel, "CUSTOMER");
}

```

```

    }
    else if (e.getSource() == billPreviousButton) {

        cardLayout.show(cardPanel, "FOOD");
    }

    else if (e.getSource() == clearButton) {

        clearAll();
    }
}

private void generateBill() {

    double subtotal = 0;

    billArea.setText("");

    billArea.append(

    );

    billArea.append(
        "Customer Name : "
        + customerNameField.getText() + "\n"
    );

    billArea.append(
        "Table Number : "
        + tableNumberField.getText() + "\n\n"
    );

    billArea.append("Item\tQty\tPrice\tTotal\n" );

    billArea.append("-----\n");

    for (int i = 0; i < items.size(); i++) {

        double total = quantities.get(i) * prices.get(i);

        subtotal += total;

        billArea.append(

```

```

        items.get(i) + "\t"
            + quantities.get(i) + "\t"
            + String.format("%.2f", prices.get(i))
            + "\t"
            + String.format("%.2f", total)
            + "\n"
    );
}

double gst = subtotal * 0.05;

double finalAmount = subtotal + gst;

billArea.append(
    "\n-----\n"
);

billArea.append("Subtotal : Rs. " + String.format("%.2f", subtotal) + "\n" );

billArea.append("GST (5%)      : Rs. " + String.format("%.2f", gst) + "\n" );

billArea.append("Final Bill   : Rs. " + String.format("%.2f", finalAmount) + "\n" );

billArea.append("\n=====");
}

private void clearAll() {

    customerNameField.setText("");
    tableNumberField.setText("");
    quantityField.setText("");

    items.clear();
    quantities.clear();
    prices.clear();

    billArea.setText("");

    cardLayout.show(cardPanel, "CUSTOMER");
}

private void showMessage(String message) {

```

```

Dialog dialog = new Dialog(this, "Message", true);

dialog.setLayout(new FlowLayout());

Label label = new Label(message);

Button ok = new Button("OK");

ok.addActionListener(e -> dialog.dispose());

dialog.add(label);
dialog.add(ok);

dialog.setSize(350, 150);
dialog.setLocation(300, 300);

dialog.setVisible(true);
}

public static void main(String[] args) {

    new FoodOrderingSystem();
}
}

```



